

TP04

Estructuras de Datos

Matias Rolón

143354

1. Codifique un script que indexe una colección¹ que requiera el volcado parcial a disco (asumiendo que existe un límite de memoria) implementando el método BSBI visto en clase. Para esto su script debe recibir un parámetro n que indica cada cuántos documentos se debe hacer el volcado a disco. Al finalizar, debe unir (merge) los índices parciales. Para las pruebas use la colección snapshot de Wikipedia² y varios valores de n (por ejemplo, $n = 10\%$ del tamaño de la colección). Registre los tiempos de indexación y de merge por separado. Grafique la distribución de tamaños de las posting lists. Calcule el overhead de su índice respecto de la colección. ¿Qué conclusiones se pueden extraer?

Respuesta

El script graba en disco los archivos *index.bin* (posting lists en binario), *vocabulary.pkl* (diccionario con el vocabulario que se carga en memoria) y *doc2file.pkl* (persiste los IDs de los documentos con su *path* asociado). El último archivo sólo es necesario para la consigna siguiente.

La colección *wiki-small* cuenta con 6043 documentos en total, por lo que se partió de ese número para probar ejecuciones³ con diferentes valores de N que representen un porcentaje aproximado de la colección.

Se logró un archivo final del índice que posee un tamaño de 38.06 Mb, frente a los 153.10 Mb que pesa la colección completa, logrando un ratio de **overhead de 0.2486**, es decir, que el índice pesa un 24% del tamaño original de la colección.

La diferencia puede deberse principalmente por ser contenido HTML, con markups y tags que quizás no se indexan o que son ampliamente repetidos en todos los archivos.

Si analizamos los tiempos, vemos que en el caso tanto de la indexación como de merge se mantienen estables independientemente del valor del parámetro N . En la indexación es esperable al procesar siempre los mismos 6043 archivos, mismos tokens y postings lists. Se esperaba que el tiempo de merge fuera mayor mientras más cantidad de chunks (menor valor de N) se crearan, debido a la cantidad de merges que tienen que hacerse y compararse para ir encontrando el menor [term_id, doc_id, freq] pero puede ser una diferencia despreciable para el

¹ Almacene docID o docID+frecuencia de acuerdo a un parámetro de entrada.

² [http://dg3rtljvitrle.cloudfront.net/wiki-small.tar.gz\(debug\)](http://dg3rtljvitrle.cloudfront.net/wiki-small.tar.gz(debug)),
<http://dg3rtljvitrle.cloudfront.net/wiki-large.tar.gz> (run)

³ El programa fue ejecutado bajo una arquitectura con 8GB de Ram y un CPU AMD A12-9720P de 4 núcleos, bajo S.O Ubuntu 24

procesador ya que esas comparaciones se miden en nanosegundos, contrario a, por ejemplo, las lecturas en disco que pueden contarse en microsegundos o más.

Valor de 'n'	Chunks generados	Tiempo indexación	Tiempo merge
120 (2)	51	160.18	17.04 s
300 (5%)	21	162.52 s	17.62 s
600 (10%)	11	157.72	16.39 s
1500 (25%)	5	166.94	16.41 s
3000 (50%)	2	164.59 s	16.30 s

Tabla 1: Tiempos para creación de índice BSBI según valor de N

Por el lado de las posting list creadas, podemos ver que se cumple nuevamente la ley de Zipf ya que hay más de trescientos mil términos que tienen frecuencia uno, es decir, aparecen una única vez en toda la colección. Luego aproximadamente sólo cuarenta mil con dos ocurrencias, y sigue decreciendo exponencialmente.

Se recortó el gráfico mostrando sólo aquellos términos con $DF < 30$ para poder visualizar mejor la distribución, pero existen unos pocos términos (comparado con los trescientos mil) con DF mayor a cien, los cuales generarán posting list muy largas.

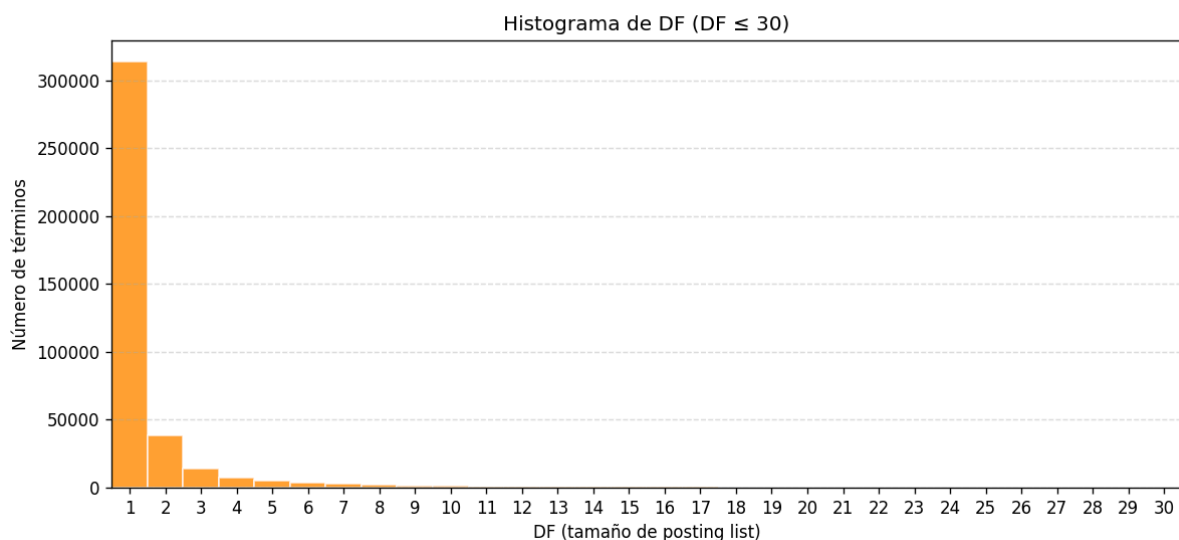


Gráfico 1: Distribución de tamaños de posting list en colección *Wiki-small*

1.1 Agregue un script que cargue el vocabulario de la colección en memoria, permita recuperar una posting completa de un término y la muestre por pantalla. Para cada documento retornado se deberá mostrar el nombre, el docID asignado durante la creación del índice y la frecuencia en el siguiente formato:

- `DocName:docID:Frecuencia`

Utilice la colección de debug para calibrar su script y verificar que la salida sea correcta.

Respuesta

Para ejecutar el programa sólo se debe indicar por parámetro el directorio donde se encontrarán los tres archivos generados por el script anterior (por defecto: `/output`).

Para esta consigna tuvimos que utilizar el documento generado `doc2file.pkl` que contiene los nombres completos de los documentos ya que en el índice sólo se almacenan sus IDs.

Al realizar un ejemplo, vemos en **Imagen 1.1** que devuelve la lista completa de documentos:

```
(venv) matias-rolon@matias-rolon-lenovo:~/workspace/2026/ri-practice/TP04/punto_1$ python3 TP04_P1_1.py output/ python
Término: 'python' | DF = 12 | term_id = 115335

DocName                                     docID  Frecuencia
-----
en/articles/c/l/a/Class (computer science).html 1191      4
en/articles/g/u/o/Guo_Youzhi_8db1.html          2231      2
en/articles/j/f/i/JFin_1830.html                 2703      2
en/articles/l/i/s/List_of Arrow Dynamics roller coasters ca82.html 3194      2
en/articles/l/i/s/List_of asteroids_103401-103500_2761.html 3244      2
en/articles/l/i/s/List_of asteroids_107601-107700_418f.html 3245      2
en/articles/p/o/n/Pontius_Pilate's_wife_80f1.html 4314      3
en/articles/r/o/x/ROX_Desktop_b4f8.html          4647      2
en/articles/s/t/e/Stellarium.html                5088      2
en/articles/w/i/l/Wilpattu_National_Park_cf60.html 5901      2
en/articles/w/i/n/Winter_solstice (disambiguation).html 5907      2
en/articles/z/e/n/Zend_Framework_a5ef.html       6014      2
-----
Total de documentos: 12
```

Imagen 1.1: Salida resultado del script `TP04_P1_1.py`

2. Codifique un script que implemente la estrategia TAAT vista en clase y sobre el índice creado en el ejercicio 1 permita realizar operaciones sobre conjuntos para buscar por dos o tres términos utilizando los operadores AND, OR y NOT.

Su script debe permitir el procesamiento de consultas del tipo:

- $((t1 \text{ AND } t2) \text{ OR } t3)$
- $((t1 \text{ AND NOT } t2) \text{ OR NOT } T3))$

Como salida su script debe retornar el nombre y el docID de los documentos que satisfacen la consulta⁴.

Respuesta

Se debe indicar al igual que el ejercicio anterior, si se ha creado el índice grabando en el archivo la frecuencia de cada término o no (parámetro `--freq`).

Se cargarán la posting list completas por cada término como un conjunto de `doc_ids`, y luego se evalúan las operaciones booleanas sobre esos conjuntos usando la biblioteca [boolean.py](https://github.com/bastikr/boolean.py) de manera recursiva, recorriendo la expresión.

⁴ <https://github.com/bastikr/boolean.py>

3. Utilizando el código e índice anteriores ejecute corridas con el siguiente subset de queries⁵ (filtre solo los de 2 y 3 términos que estén en el vocabulario de su colección) y mida el tiempo de ejecución en cada caso. Para ello, utilice los siguientes patrones booleanos:

a) Queries $|q| = 2$

- $t1 \text{ AND } t2$
- $t1 \text{ OR } t2$
- $t1 \text{ NOT } t2$

b) Queries $|q| = 3$

- $t1 \text{ AND } t2 \text{ AND } t3$
- $(t1 \text{ OR } t2) \text{ NOT } t3$
- $(t1 \text{ AND } t2) \text{ OR } t3$

¿Puede relacionar los tiempos de ejecución con los tamaños de las listas? (pruebe con el índice en disco o cargándose completamente en memoria antes). ¿Qué conclusiones se pueden extraer?

Respuesta

Partiendo del código del punto anterior, se le agrega:

- Un parámetro opcional `--in-memory`, para cargar el archivo binario del índice en memoria (por defecto, no).
- Carga del archivo `EFF-10K-queries.txt` y filtrado de todas las queries que tengan 2 o 3 términos y todos estén en el vocabulario.
- Agrega medición de tiempos y cálculos (promedios, largo de postings, etc.) alrededor del proceso que realiza la query.
- Se guarda un archivo con todas las queries realizadas y sus tiempos para poder realizar un análisis más minucioso luego.

Para éste análisis se menciona el concepto de postings “totales”, que es la suma de los largos de las posting list de cada término en una query, como referencia de la cantidad de *postings* a procesar. Ejemplo: si las postings de la query “casa perro” tienen 4 y 6 docs_ids en su lista respectivamente, esa query tendrá un total de postings de 10.

Podemos ver en **Tabla 3** que los tiempos en memoria son más cortos. Teniendo una diferencia de aproximadamente un segundo en ambos casos, tanto para dos como para tres términos.

A su vez, lógicamente los tiempos de las queries que poseen tres términos son un poco mayores a las que tienen dos, tomando en promedio, el doble de tiempo.

⁵ https://github.com/tolosoft-academia/RI_2025/blob/main/data/EFF-10K-queries.txt

Vemos que el largo total de posting a procesar también aumenta considerablemente, pasando de 323 en promedio, a 845.

	2 términos		3 términos	
Índice	En memoria	En disco	En memoria	En Disco
Queries	5793	5793	3141	3141
Posting totales promedio	323	323	835	835
Tiempo total	1.4358 s	2.3886 s	1.6531 s	2.5260 s
Tiempo promedio	0.0002 s	0.0004 s	0.0005	0.0008 s
Tiempo máximo	0.0050 s	0.0060 s	0.1422 s	0.1772

Tabla 3: tiempos para queries TAAT de 2 y 3 términos.

También podemos analizar el documento *query_results.csv* que se crea al finalizar la ejecución del programa con los tiempos individuales de cada query, viendo así los patrones más claramente:

tiempo segun largo de posting list

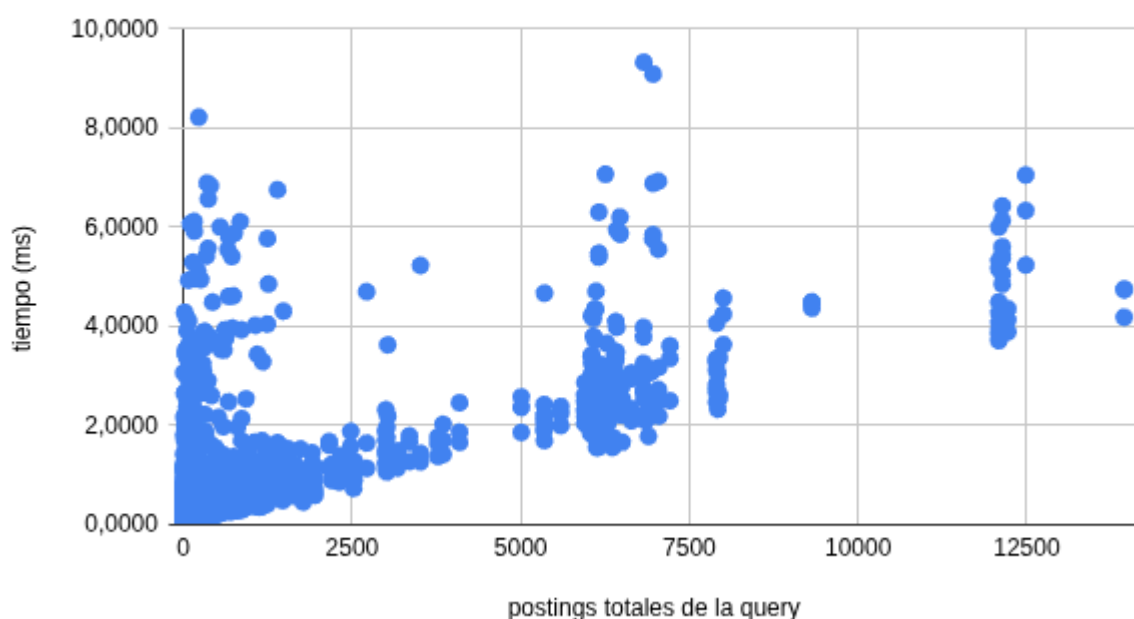


Gráfico 3.1: Tiempo de ejecución según postings totales de la query (en disco)

Guiándonos por el **Gráfico 3**, podemos afirmar que a medida que los postings totales a recuperar son mayores, aumenta también el tiempo total de resolución de la consulta, medido en este caso en milisegundos (ms). Aunque el mayor volumen de queries se ubica dentro de las menores a 2000 postings con un tiempo entre 0 y 2 ms. Luego, los términos con más de 2000 postings se ubican arriba de los 2 ms.

Esto se explica ya que TAAT carga las posting list enteras de cada término, y mientras más postings tengan, serán más elementos para intersectar/unir (dependiendo si es AND u OR).

Otro gráfico que podemos hacer con la información del csv resultante, es el de los tiempos según el patrón de la consulta:



Gráfico 3.2: Tiempo según patrón de la consulta

Pudiendo observar que la que menos tiempo tarda es el OR simple, a pesar de ser seguramente la que más documentos nos de como resultado, quizás optimizado por la función set() de python. Luego, la que más tarda es la query AND que contiene un OR y un NOT en sus subqueries: (t1 OR t2) AND NOT t3.

A su vez, también podemos confirmar la diferencia entre consultas de dos y tres términos indicados anteriormente en la **Tabla 3**, ya que los primeros son los de tiempo más bajo (menores a 0.5 ms) sin importar el patrón.

4. Codifique un script que implemente la estrategia DAAT vista en clase y sobre el índice creado en el ejercicio 1 permita resolver consultas usando el modelo vectorial utilizando la métrica del coseno como medida de similitud. Dada una consulta su script debe retornar los top-k documentos de score mayor.

Su script debe mostrar como salida el nombre, el docID y el score (ordenado por score) con el siguiente formato

- DocName:docID:Score

Respuesta:

Se abren las posting lists de todos los términos de la query y se recorren todas por docID ascendente. Para cada documento se computa el score completo usando DAAT con diferencia de coseno para el ranking, manteniendo un min-heap de tamaño k (parámetro del programa) para devolver los mejores.

5. Agregue skip lists a su índice del ejercicio 1 y ejecute un conjunto de consultas AND sobre el índice original y luego usando los punteros. Compare los tiempos de ejecución con los del ejercicio 3.

Respuesta

Manning[2] explica que para una posting list de largo P, una heurística simple que funciona bien en la práctica es usar $\sqrt{P}$ skip pointers equidistantes, por lo que se implementó esta estrategia.

El script tomará el índice actual y construirá un archivo auxiliar de skip lists (*skips.bin*). Lo usará para realizar las queries que se piden en la consigna, grabandolo a disco (archivo *skips.bin*). El punto siguiente da por sentado que ya se tiene una estructura de skips list creada.

Además, se agrega al *vocabulary.pkl* (creado ya en el punto 1) un dato extra para cada término, que indica la posición en el archivo nuevo de skip lists dónde empezar a leerlo. En caso de ser agregado el dato por primera vez, se sobrescribe el archivo del vocabulario para que persista en disco con su nueva columna.

	2 términos		3 términos	
Índice	Sin skips	Con skips	Sin skips	Con skips
Queries	1931	1931	1047	1047
Posting totales promedio	323	323	835	835
Tiempo total	0.125141 s	0.126147 s	0.123322 s	0.082826 s
Tiempo promedio	0.000065 s	0.000065 s	0.000118 s	0.000079 s
Tiempo máximo	0.002016 s	0.002439	0.003393 s	0.005054 s

Tabla 5: tiempos para queries de 2 y 3 términos, con y sin skip pointers.

Si observamos los tiempos medidos, los resultados de las consultas con dos términos son muy similares, no habiendo una diferencia sustancial para decir que una implementación es mejor que otra (con skips vs sin skips). Sin embargo, ya con consultas de tres términos la diferencia se amplía. El tiempo total pasa de 0.12 a 0.08 usando skip list, lo que equivale a una mejora de casi el 50%. No se lo

relaciona tanto a la cantidad de términos en sí, si no al posible aumento en el largo de postings de alguno de esos 3 términos, que es lo que realmente hace la diferencia, ya que los skips harán saltar varias posiciones cuando se esté realizando la intersección entre postings largas.

Se genera un archivo *query_results_skip.csv* con los tiempos de todas las consultas individuales. Si filtramos por aquellas ejecuciones con skip lists, vemos que los tiempos están muchos más concentrados que en el del punto 3. Salvo contadas excepciones, la mayoría de los casos no supera el segundo, sin diferenciar por el largo de las posting lists. Un ejemplo de ello es el sector mayor a 5000 postings, donde sólo unos pocos superan los 2 ms, a comparación de ese mismo sector en el **Gráfico 3.1** donde todos los puntos se ubicaban por encima de esa línea de 2 ms.

Tiempo segun cant. de postings usando skip lists

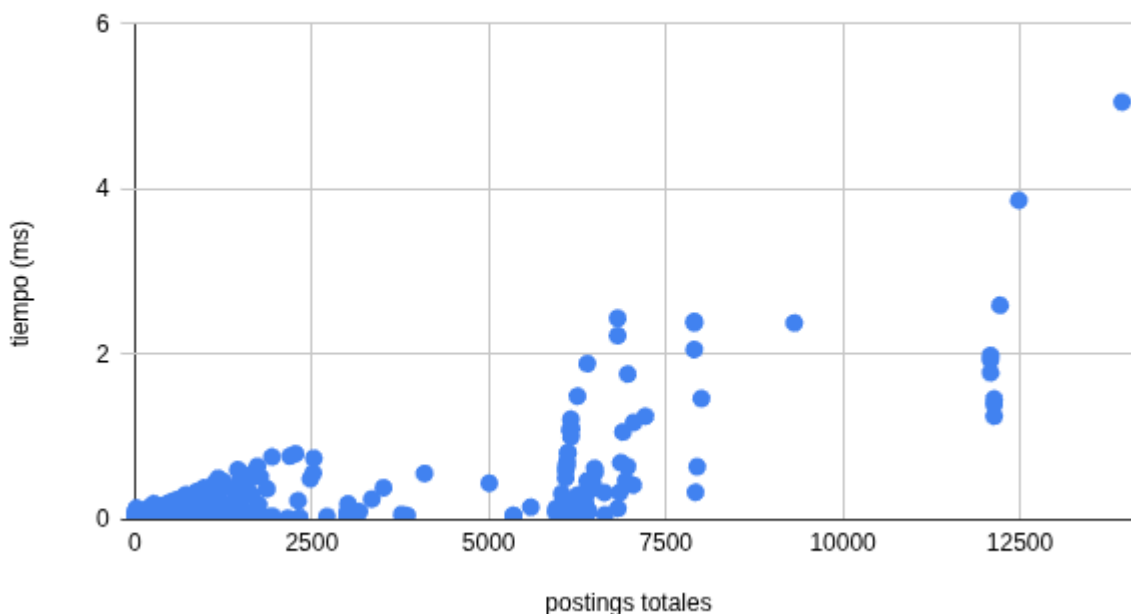


Gráfico 5: Tiempo de ejecución según postings totales de la query (con skip lists)

5.1 Agregue un script que permita recuperar las skips list para un término dado. En este caso la salida deberá ser una lista de docName:docID ordenada por docName.

Respuesta

Nos apoyaremos en el nuevo dato *skip_seek* para poder empezar la búsqueda de las skips del término pasado por parámetro.

Se deberá pasar por parámetro nuevamente la carpeta donde se encuentra el índice original. Por el lado de la obtención del nuevo archivo (*skips.bin*) siempre lo buscará en la carpeta */output* del punto actual.


```

• (venv) matias-rolon@matias-rolon-lenovo:~/workspace/2026/ri-practice/TP04/punto_5$ python3 TP04_P5_1.py ../punto_1/output/
python
Término 'python' (df=12, 4 skip pointers):
en/articles/c/l/a/Class (computer science).html:1191
en/articles/l/i/s/List of Arrow Dynamics roller coasters_ca82.html:3194
en/articles/p/o/n/Pontius Pilate's wife_80f1.html:4314
en/articles/w/i/l/Wilpattu National Park_cf60.html:5901

```

Imagen 5.1: Salida resultado del script *TP04_P5_1.py*

Haciendo una prueba rápida, busquemos el mismo término que en el ejemplo del punto 1: python. Vemos que obtenemos la misma información de DF=12, y que los skip pointers mostrados corresponden a documentos que ya fueron mostrados en la **Imagen 1.1**, por lo que la información devuelta es consistente.

6. Sobre la colección Dump10k⁶ Escriba un programa que realice una evaluación TAAT y otro usando DAAT. Compare los tiempos de ejecución para un conjunto de queries dados⁷. Separe su análisis por longitud de queries y de posting lists.

Respuesta

La colección Dump10k ya es un índice invertido en estructura, por lo que se lo carga en memoria directamente y luego se realizan las queries. Ya que el objetivo es comparar los tiempos de las distintas estrategias de búsqueda. En este caso, ambas irán a buscar las postings a memoria, por lo que estarían en iguales condiciones iniciales.

Si se quisiera aplicar el enfoque de puntos anteriores, se debería convertir el archivo .txt a un binario mediante algún script intermedio.

Análisis por longitud de queries, con tiempo:

Cant. términos query	TAAT			DAAT		
	t. total	t. prom	t. max	t. total	t. prom	t. max
q = 1	973 ms	0.38 ms	58 ms	159 ms	0.06 ms	7 ms
q = 2	5.5 s	2.20 ms	1.7 s	5.7 s	2.31 ms	2.7 s
q = 3	3.5 s	1.39 ms	100 ms	383 ms	0.15 ms	100 ms
q = 4	4.3 s	1.74 ms	253 ms	41 ms	0.01 ms	1 ms

Tabla 6.1: Tiempos según cantidad de términos de la query (TAAT vs DAAT)

DAAT domina ampliamente excepto en q=2 donde son muy similares en tiempo.

⁶ <http://www.tyr.unlu.edu.ar/tallerIR/2014/data/dump10k.tar.gz>

⁷

<http://www.labredes.unlu.edu.ar/sites/www.labredes.unlu.edu.ar/files/site/data/ri/queriesDump10K.txt.tar.gz>

Esta diferencia de tiempo se va ampliando a medida que la query es más larga. Si se realiza el cálculo $t. TAAT / t. DAAT$ para saber la diferencia de rendimiento empezamos en $q=1$ con un factor de 6.11x, es decir, que DAAT es 6 veces más rápido que TAAT.

Luego en $q=2$ es de 0.95x, lo que indica que TAAT fue ligeramente más rápido, teniendo tiempos prácticamente iguales.

Apartir de $q=3$ vemos una mejora sustancial en los tiempos DAAT siendo 9.12x veces más rápido, pudiendo explicarse a partir de como hace sus intersecciones, ya que luego de la primera se va achicando la cantidad de documentos constantemente, siendo en cada intersección con cada documento un resultado parcial del conjunto de documentos final, y sin necesitar conversiones ni otras estructuras, ya que usa la posting list ordenada del índice. En cambio TAAT arma las listas (convierte a sets de python en este caso) con todos los términos antes de hacer la intersección, los ordena y paga el costo de buscar la totalidad de los docIDs de cada posting list sin importar si la mayoría se descarta luego en la intersección final. Por ejemplo, dado una query con "t1 t2 t3 t4" si la primera intersección entre postings de t1 y t2 da vacío, igual se desperdició tiempo convirtiendo a sets y ordenando todas las postings de los términos 3 y 4.

Esto se confirma una vez más en $q=4$ donde el rendimiento llega a ser de 106 veces mejor en DAAT.

Finalmente, analizamos por longitud de posting list con los datos de la **Tabla 6.2**. Teniendo en cuenta la distribución de tamaños de las posting lists se calculan tres partes iguales dinámicamente discriminando chicas (<29), medias (>30 y <418) y grandes (>418) para esta colección.

Vemos que DAAT es consistentemente más rápido en las tres categorías. Sin embargo, a medida que la posting list es más larga, esa diferencia es menos pronunciada, ya que para posting lists chicas es de 4,4x, pasando por 4,2x en medias y por último 2.2x en grandes. Esto hace deducir que para la intersección entre posting list cada vez más largas, pesa más el overhead que le agrega python sea el código que sea, que la estrategia que se implemente en sí, por lo que hay que optimizarlo si se quiere trabajar con sets, hacer ordenamientos, etc.

Largo posting list	TAAT			DAAT		
	t. total	t. prom	t. max	t. total	t. prom	t. max
chica	341 ms	0.10 ms	71 ms	76 ms	0.02 ms	7 ms
media	290 ms	0.08 ms	57 ms	68 ms	0.02 ms	6 ms
grande	13.7 s	4.11 ms	1.7 s	6.2 s	1.86 ms	2.7 s

Tabla 6.2: Tiempos según largo de las posting list (TAAT vs DAAT)

7. Comprima el índice del ejercicio 1 utilizando Variable-Length Codes (VByte) para los docIDs y Elias-gamma para las frecuencias (almacene docIDs y frecuencias en archivos separados). Calcule tiempos de compresión/descompresión del índice completo y tamaño resultante en cada caso. Realice dos experimentos, con y sin DGaps. Compare los tamaños de los índices resultantes.

7.1 Agregue un script que permita recuperar de disco la posting list de un término dado y la versión comprimida de dicha lista.

Respuesta

Tamaño del índice original: **24.18 Mb**

	Sin Dgaps	Con Dgaps
Tamaño doc ids (vbyte)	5.97 Mb	3.74 Mb
Tamaño freqs (Elias Gamma)	1.19 Mb	1.19 Mb
Total comprimido	7.16 Mb	4.93 Mb
Tiempo de compresión	24.91 seg.	23.52 seg.

Tabla 7.1: estadísticas de compresión con y sin dgaps

Vemos que los tamaños tienen sentido con el script, ya que sólo se aplicó dgaps al archivo de los docs id. Por esta razón el archivo de frecuencias no cambia su tamaño independientemente de que se implementen gaps o no.

De esta manera, tenemos un ratio del 29.6% con el método sin dgaps, lo que nos indica que la compresión resultó en casi $\frac{1}{3}$ del tamaño original del índice, versus un ratio del 20.4% con dgaps, que logra hacerlo en $\frac{1}{5}$ de ese tamaño original, por ende conviene utilizar gaps en este caso. Los valores a comprimir se vuelven mucho más chicos ya que pasan a ser deltas entre docs_id consecutivos (un doc_id será igual a la diferencia con el doc_id anterior).

Además, si bien también se logró un menor tiempo total de compresión con este último método, la diferencia es mucho menor con respecto a la vista en los tamaños, siendo de 1.5 segundos. Aún así cabe remarcar este punto, ya que para colecciones mucho mayores puede volverse una diferencia significativa (5% del tiempo aprox).

Por último, al intentar obtener un término junto con su posting list original descomprimida (TP04_P7_1.py) vemos los siguientes tiempos para algunos ejemplos al azar:

		Índice original		Índice comprimido	
Término	DF	Tamaño	Lectura	Tamaño	Lectura
politic	6	48 bytes	3.2621	12 bytes	0.6003 ms
contamination	15	120 bytes	2.9298 ms	29 bytes	0.5672 ms
dragon	43	344 bytes	3.1757 ms	74 bytes	0.7681 ms
america	250	2000 bytes	3.1817	320 bytes	0.8028 ms
house	427	3416 bytes	0.6006 ms	563 bytes	1.2235 ms
reference	1831	14648 bytes	1.5858 ms	2689 bytes	3.4013 ms
table	5345	42760 bytes	5.7978 ms	8701 bytes	9.1871 ms

Tabla 7.1: Tiempos de lectura índice original vs comprimido

Siempre se leen postings de menor tamaño en el índice comprimido. También suele tener mejor tiempo para procesar los binarios, pero cuando empezamos a probar con términos que tienen una cantidad mayor de docs id en sus posting (mayor DF) vemos que esos tiempos empiezan a cambiar radicalmente. Puede deberse al tiempo de CPU que requiere desencodear los valores con el método variable byte y elías gamma, que se hace bit a bit, y para muchos más números.

Bibliografía

- [1] Bruce Croft, Donald Metzler, and Trevor Strohman. Search Engines: Information Retrieval in Practice. Addison-Wesley Publishing Company, USA, 1st edition, 2009.
- [2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, New York, NY, USA, 2008.
- [3] Zobel, J. y Moffat, A. Inverted Files for Text Search Engines. RMIT University & University of Melbourne, 2006.